HomeLab Monitoring

Documentation d'installation complète

Stack	Matériel	Durée totale	Phases
Prometheus · Grafana · Uptime Kuma	Raspberry Pi (arm64 / armv7)	~4 heures	4

Ce document est ta référence pas-à-pas pour monter un stack de monitoring complet sur Raspberry Pi. Chaque étape inclut les commandes exactes à copier-coller, les vérifications à faire, et un emplacement pour coller ton screenshot.

Prérequis

Matériel & OS

- Raspberry Pi 4 ou 5
- Raspberry Pi OS 64-bit installé et démarré
- Connexion SSH ou accès clavier/écran
- Accès réseau local (le Pi et PC sur le même réseau)

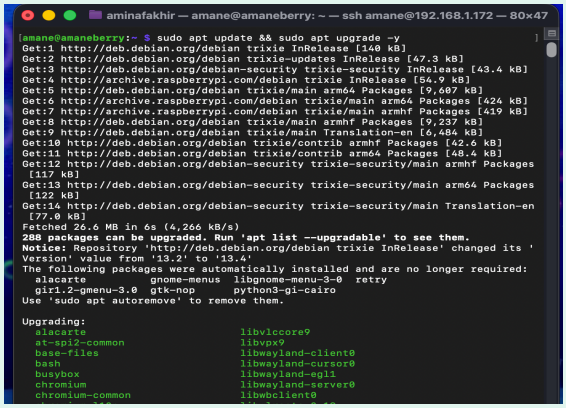
Architecture du projet

Composant	Rôle	Port
node_exporter	Collecte les métriques système	9100
Prometheus	Stocke les métriques	9090
Grafana	Visualise & alerte	3000
Uptime Kuma	Surveillance disponibilité	3001

Phase 1 — Prérequis système & Prometheus

Étape 1.1 — Mise à jour du système

```
sudo apt update && sudo apt upgrade -y
```



Étape 1.2 — Télécharger et installer Prometheus

Téléchargement de la dernière version de Prometheus pour l'architecture arm64 (Pi 4/5) :

```
cd /tmp
wgt https://github.com/prometheus/prometheus/releases/download/v3.11.1/prometheus-3.11.1.linux-arm64.tar.gztar xvf prometheus-*.linux-arm64.tar.gz
tar xvf prometheus-3.11.1.linux-arm64.tar.gz
sudo mv prometheus-3.11.1.linux-arm64/prometheus /usr/local/bin/
sudo mv prometheus-3.11.1.linux-arm64/promtool /usr/local/bin/
```

Vérifier l'installation :

```
prometheus --version
```

```
[amane@amaneberry:/tmp $ prometheus --version
prometheus, version 3.11.1 (branch: HEAD, revision: 1bd2f3a9fdedf52e6f613449cc4c
50e86ca24676)
  build user:       root@b1d74f9af457
  build date:       20260407-15:07:44
  go version:       go1.26.1
  platform:         linux/arm64
  tags:             netgo,builtinassets
amane@amaneberry:/tmp $
```

Étape 1.3 — Configurer prometheus.yml

Créer le dossier de configuration et le fichier principal :

```
sudo mkdir -p /etc/prometheus
sudo mkdir -p /var/lib/prometheus
sudo nano /etc/prometheus/prometheus.yml
```

Contenu du fichier prometheus.yml :

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: 'node_local'
    static_configs:
      - targets: ['localhost:9100']
```

Étape 1.4 — Créer le service systemd Prometheus

Créer l'unité systemd pour démarrer Prometheus automatiquement :

```
sudo nano /etc/systemd/system/prometheus.service
```

Contenu du fichier :

```
[Unit]
Description=Prometheus Monitoring
```

```
After=network.target

[Service]
ExecStart=/usr/local/bin/prometheus \
  --config.file=/etc/prometheus/prometheus.yml \
  --storage.tsdb.path=/var/lib/prometheus
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Activer et démarrer le service :

```
sudo systemctl daemon-reload
sudo systemctl enable prometheus
sudo systemctl start prometheus
sudo systemctl status prometheus
```

```
sudo systemctl status prometheus
Created symlink '/etc/systemd/system/multi-user.target.wants/prometheus.service'
 → '/etc/systemd/system/prometheus.service'.
● prometheus.service - Prometheus Monitoring
     Loaded: loaded (/etc/systemd/system/prometheus.service; enabled; preset: e>
     Active: active (running) since Sun 2026-04-12 20:38:18 CEST; 64ms ago
 Invocation: d8b3d9919b84443da7e08aa67853e3c0
   Main PID: 29368 (prometheus)
      Tasks: 5 (limit: 3913)
        CPU: 32ms
     CGroup: /system.slice/prometheus.service
             └─29368 /usr/local/bin/prometheus --config.file=/etc/prometheus/pr>
```

Phase 2 — Node Exporter — métriques système ~30 min

Étape 2.1 — Télécharger et installer node_exporter

Node Exporter expose les métriques système (CPU, RAM, disque, réseau) que Prometheus va collecter.

```
cd /tmp
wget
https://github.com/prometheus/node_exporter/releases/download/v1.11.1/node_exporter-
1.11.1.linux-arm64.tar.gz
tar xvf node_exporter-1.11.1.linux-arm64.tar.gz
sudo mv node_exporter-1.11.1.linux-arm64/node_exporter /usr/local/bin/
```

▶️ Étape 2.2 — Créer le service systemd node_exporter

```
sudo nano /etc/systemd/system/node_exporter.service
```

```
[Unit]
Description=Node Exporter
After=network.target

[Service]
ExecStart=/usr/local/bin/node_exporter
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable node_exporter
sudo systemctl start node_exporter
sudo systemctl status node_exporter
```

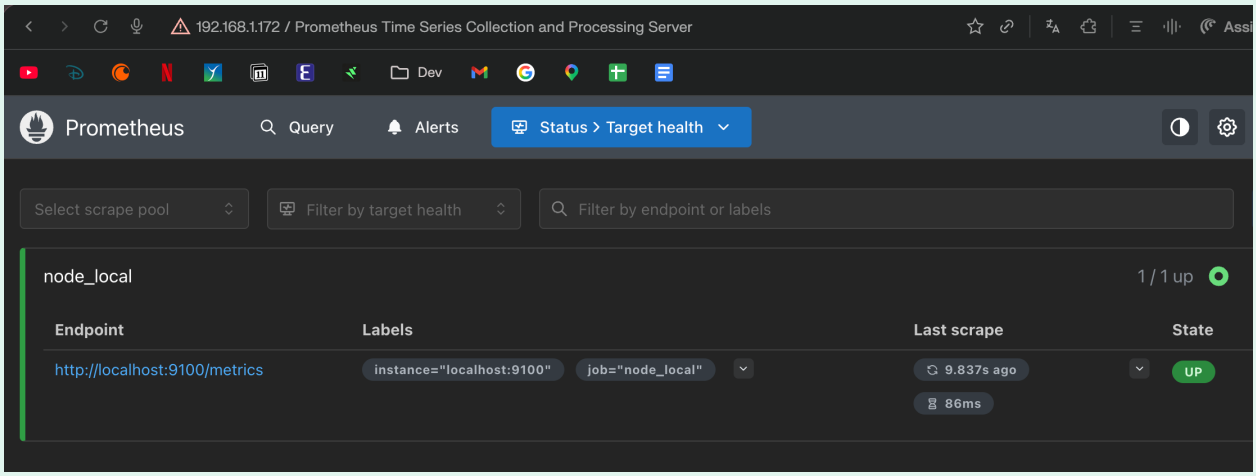
Étape 2.3 — Vérifier que Prometheus scrape les métriques

Tester si les métriques sont exposées :

```
curl http://localhost:9100/metrics | head -20
```

Vérifie dans l'interface Prometheus que le target est UP :

```
# Ouvrir dans le navigateur
http://192..168.1.172:9090/targets
```



Phase 3 — Grafana — Dashboard & Alertes

Étape 3.1 — Installer Grafana

Ajouter le dépôt officiel Grafana et installer la version stable :

```
sudo mkdir -p /etc/apt/keyrings
wget -q -O - https://apt.grafana.com/gpg.key | gpg --dearmor | sudo tee
/etc/apt/keyrings/grafana.gpg > /dev/null
echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg] https://apt.grafana.com stable
main" | sudo tee /etc/apt/sources.list.d/grafana.list
sudo apt update && sudo apt install -y grafana
```

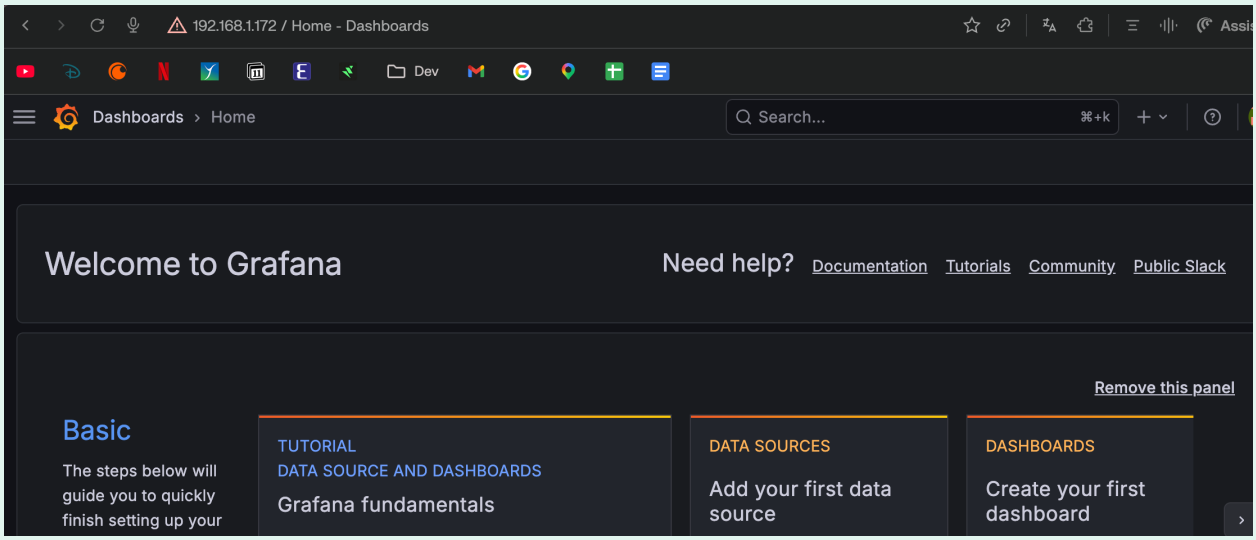
```
sudo systemctl daemon-reload
sudo systemctl enable grafana-server
sudo systemctl start grafana-server
sudo systemctl status grafana-server
```

Étape 3.2 — Premier accès et connexion

Ouvrir Grafana dans le navigateur :

```
http://192.168.1.172:3000
```

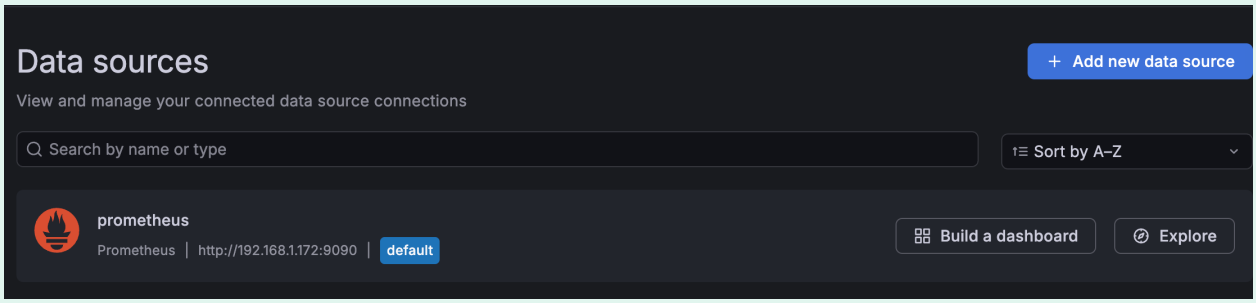
- Login par défaut : admin
- Mot de passe par défaut : admin
- Grafana demande de changer le mot de passe à la première connexion



Étape 3.3 — Connecter Prometheus comme Data Source

Dans Grafana, ajouter Prometheus comme source de données :

- **Connections → Data Sources**
- **Add data source**
- **Prometheus**
- Remplir le champ URL
- **Save & Test**



Étape 3.4 — Importer le dashboard Node Exporter Full

Ce dashboard prêt-à-l'emploi offre immédiatement CPU, RAM, réseau, température, disque :

- **Dashboards → Import**
- **ID : 1860**
- **Load**
- Dans le menu déroulant Prometheus, sélectionner la source de données
- **Import**

Étape 3.5 — Créer une alerte CPU

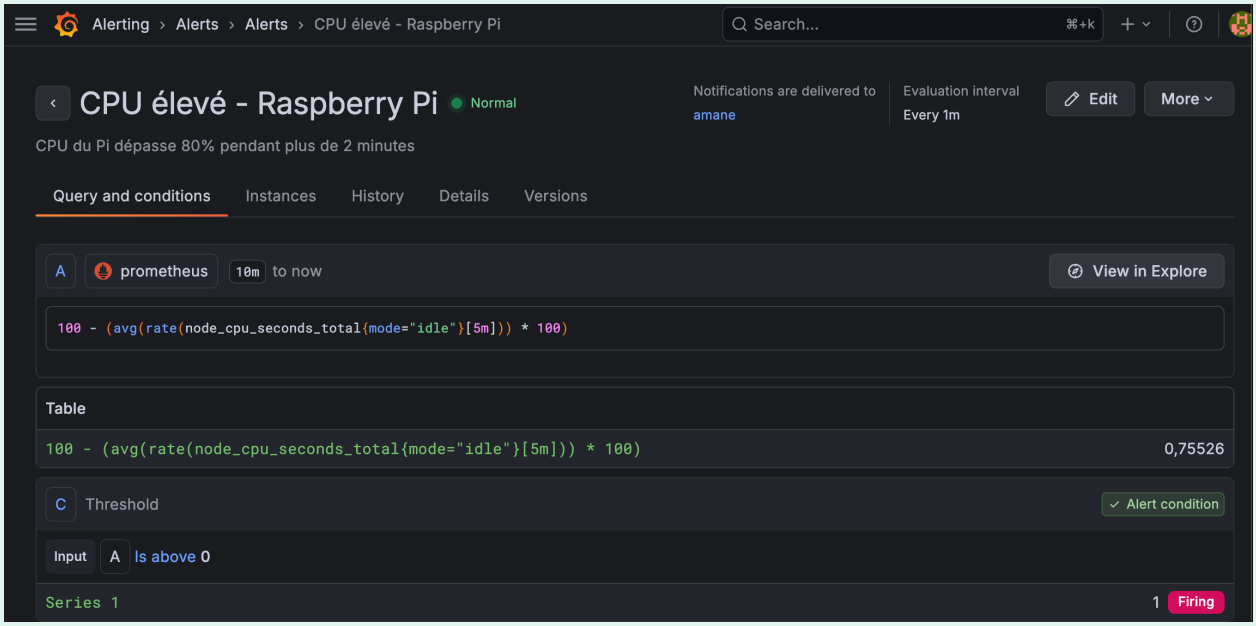
Création d'une règle d'alerte qui se déclenche quand le CPU dépasse 80% pendant 2 minutes :

- **Alerting → Alert Rules → New alert rule**
- **Nom règle : CPU élevé — Raspberry Pi**

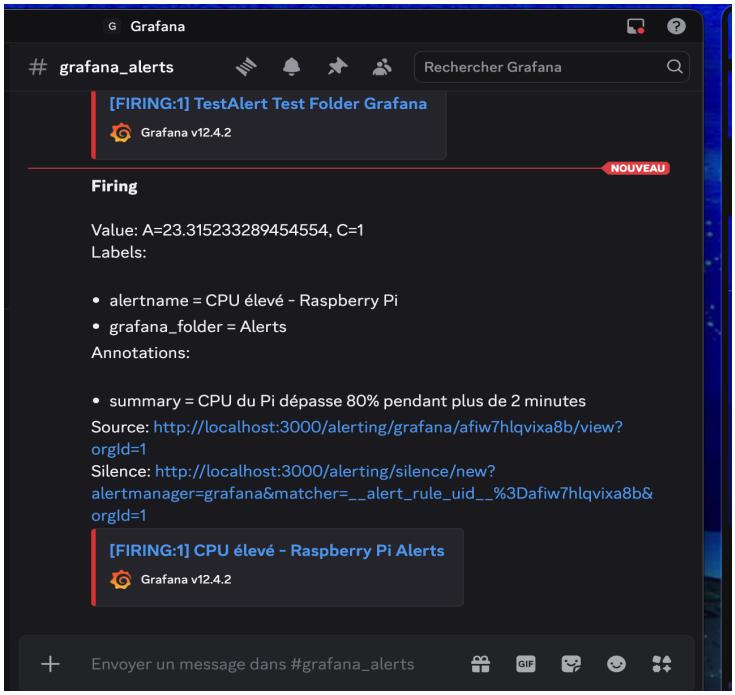
Dans la section Query, entrer cette requête PromQL :

```
100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)
```

- Set the threshold : IS ABOVE 80
- Evaluate every : 1m For : 2m
- Summary : CPU du Pi dépasse 80% pendant plus de 2 minutes



Utilisation d'un webhook Discord comme contact point (ne nécessite aucune configuration système). Les alertes sont envoyées directement sur un serveur discord dédié.



Phase 4 — Uptime Kuma — Surveillance de disponibilité

Uptime Kuma surveille la disponibilité des services (Grafana, Prometheus,portfolio, etc.) et génère un beau tableau de bord de statut.

Étape 4.1 — Installer Node.js 20

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
node --version
```

Étape 4.2 — Cloner et configurer Uptime Kuma

```
sudo git clone https://github.com/louislam/uptime-kuma.git /opt/uptime-kuma
cd /opt/uptime-kuma
sudo npm run setup
```

Étape 4.3 — Créer le service systemd

```
sudo nano /etc/systemd/system/uptime-kuma.service
```

```
[Unit]
Description=Uptime Kuma
After=network.target

[Service]
WorkingDirectory=/opt/uptime-kuma
ExecStart=/usr/bin/node server/server.js
Restart=always
RestartSec=5
Environment=NODE_ENV=production

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable uptime-kuma
sudo systemctl start uptime-kuma
sudo systemctl status uptime-kuma
```

Étape 4.4 — Configurer les monitors

Accèder à Uptime Kuma :

```
http://192.168.1.172:3001
```

Créer un compte admin à la première connexion, puis ajouter ces monitors :

Nom	Type	URL / Host	Intervalle
Grafana	HTTP(s)	http://localhost:3000	60s
Prometheus	HTTP(s)	http://localhost:9090	60s
Mon portfolio	HTTP(s)	https://amanefk.github.io/portfolio-bts/	120s

Uptime Kuma
Pages de statut
Tableau de bord
+ Ajouter une nouvelle sonde
État
Étiquettes
Rechercher...
100% Grafana
100% Portfolio
100% Prometheus
Grafana #1
http://192.168.1.172:3000/
En pause
Modifier
Cloner
Supprimer
En ligne
Vérifier toutes les 60 secondes (1 minute)
Temps de réponse
(Actuellement)
55ms
Réponse moyenne
(24 heures)
53.75ms
Disponibilité
(24 heures)
100%
Disponibilité
(30 jours)
100%

Vérification finale

Une fois toutes les phases terminées, vérifie que tout tourne ensemble :

```
sudo systemctl status prometheus node_exporter grafana-server uptime-kuma
```

> 💡 *Les 4 services doivent afficher 'active (running)'. Si l'un est en erreur, consulte les logs avec : sudo journalctl -u <nom-service> -n 50*

📸 **Screenshot : statut des 4 services — tous active (running)**